



TA Playbook: Building the XAI Evaluation Kit

GitHub Repo: github.com/RAI-Incubation-Lab/AXAI-Toolkit

Everything below gets checked into this repo

Your Role

You are the **Product Manager and DevOps Engineer** of this course. You build the infrastructure so that inexperienced student class teams face zero environment friction. They copy your templates, plug in their AI solution and test data, and get explainability results.

Repo Structure: Where Everything Goes

```
AXAI-Toolkit/                                ← github.com/RAI-Incubation-Lab/AXAI-Toolkit
├── .github/
│   └── PULL_REQUEST_TEMPLATE.md             ← Deliverable C
├── templates/
│   ├── AXAI_Master_Template.ipynb          ← Deliverable A (Colab notebook)
│   ├── model_card_template.md              ← Deliverable B1
│   ├── checklist_template.md               ← Deliverable B2
│   ├── grading_rubric.md                   ← Deliverable C (for instructor)
│   └── README.md                           ← Deliverable E (student setup guide)
├── tools/
│   └── modelcard/                           ← Deliverable F (parallel track)
│       ├── __init__.py
│       ├── generate.py
│       └── fairness.py
├── Case_Studies/
│   └── demo-titanic/                        ← Deliverable D
│       ├── demo-titanic.ipynb
│       ├── MODEL_CARD.md
│       ├── README.md
│       ├── app.py
│       ├── demo.mp4
│       └── requirements.txt
└── README.md                               ← Main repo landing page
```

Deliverables Overview

#	Deliverable	Where in Repo
A	One-Click Colab Master Template	templates/AXAI_Master_Template.ipynb
B	Evaluation Markdown Templates	templates/model_card_template.md , templates/checklist_template.md

#	Deliverable	Where in Repo
C	GitHub Grading Pipeline & PR Template	<code>.github/PULL_REQUEST_TEMPLATE.md</code> , <code>templates/grading_rubric.md</code>
D	Demo Repo (complete worked example)	<code>Case_Studies/demo-titanic/</code>
E	Setup Guide for students	<code>templates/README.md</code>
F	Model Card Generator Package	<code>tools/modelcard/</code>

Minimum viable: A + B + C. Students can complete the project with just the template notebook and markdown templates. Deliverables D and E improve the experience but aren't required to start. Deliverable F runs on a parallel track.

Deliverable A: One-Click Colab Master Template

File: `templates/AXAI_Master_Template.ipynb`

A boilerplate Jupyter notebook. Students open it, run from top to bottom, and get explainability results for **any AI solution** — whether they trained a model themselves, used an API (OpenAI, Claude), or used a no-code AI tool.

Required Cells (in order)

Cell 1: Master Import Cell

```
# =====
# RAI INCUBATION LAB - MASTER TRUST STACK IMPORT CELL
# =====
# Run this cell first. Installs all required libraries automatically.
# Takes ~2-3 minutes on first run.
!pip install -q shap lime interpret fairlearn aif360 streamlit gradio
```

Cell 2: Solution Description (student fills this in)

```
# =====
# STEP 1: DESCRIBE YOUR AI SOLUTION
# Fill in the details below about the AI solution you're evaluating.
# =====

# Solution info
solution_name = "Your Project Name"          # TODO: change
solution_type = "classification"              # classification | regression | text_generation
tech_stack = "ChatGPT API + custom prompts"   # TODO: describe tools/APIs/models used
dataset_description = "Loan application dataset" # TODO: describe your data
target_variable = "loan_approved"             # TODO: what does your solution predict?

# API key (if using cloud AI services)
# IMPORTANT: Use environment variables, don't hardcode keys
# import os
# os.environ["OPENAI_API_KEY"] = getpass("Enter API key: ")
```

Cell 3: Load Test Data (student replaces this section)

```
# =====
# STEP 2: LOAD YOUR TEST DATA
# Replace with your own dataset. The template expects a CSV with features
# and a target column. Adjust as needed for your solution.
# =====

import pandas as pd

# TODO: Replace with your dataset path and target column
user_dataset = pd.read_csv("your_data.csv")
target_column = "your_target_column_name"

# Split features and target
X = user_dataset.drop(columns=[target_column])
y = user_dataset[target_column]

print(f"Loaded {len(X)} samples with {len(X.columns)} features")
```

Cell 4: Get Predictions from Your Solution

```
# =====
# STEP 3: GET PREDICTIONS FROM YOUR SOLUTION
# This cell collects predictions so explainability tools can analyze them.
# Choose ONE path based on your solution type.
# =====

# --- PATH A: You have a Python model (scikit-learn, XGBoost, etc.) ---
predictions = user_model.predict(X)          # predicted classes
probabilities = user_model.predict_proba(X)    # prediction confidence

# --- PATH B: You use an API (OpenAI, Claude, etc.) ---
# Write a function that sends each row to your API and returns the prediction
# def call_ai_api(row):
#     prompt = f"Based on these inputs, predict the outcome: {row.to_dict()}"
#     response = openai.chat.completions.create(messages=[{"role": "user", "content": prompt}])
#     return response.choices[0].message.content
#
# predictions = X.apply(call_ai_api, axis=1)
# probabilities = None # APIs may not return confidence scores

# --- PATH C: You use a no-code tool ---
# Export predictions from your tool as a CSV, then load:
# predictions = pd.read_csv("predictions.csv")["predicted_column"]

print(f"Collected {len(predictions)} predictions")
```

Cell 5: SHAP Analysis (runs automatically)

```
# =====
# STEP 4: SHAP EXPLAINABILITY ANALYSIS
# This cell runs automatically. Shows which features drive predictions.
# Do not modify unless you know what you're doing.
# =====

import shap
import numpy as np

# For tree-based models (PATH A with RandomForest/XGBoost)
try:
    explainer = shap.TreeExplainer(user_model)
    shap_values = explainer.shap_values(X)
    print("Using TreeExplainer (fast)")

# For other models, fall back to KernelExplainer (works with any model)
except:
    print("Falling back to KernelExplainer (slower but universal)...")
    background = shap.sample(X, 100) # sample for speed
    explainer = shap.KernelExplainer(lambda x: user_model.predict(x), background)
    shap_values = explainer.shap_values(X.iloc[:50]) # analyze subset

# Global feature importance
print("\n=== Global Feature Importance (SHAP) ===")
shap.summary_plot(shap_values, X, show=False)

# Local explanation for first sample
print("\n=== Local Explanation: Sample #1 ===")
if hasattr(shap, 'Explanation'):
    shap.waterfall_plot(shap.Explanation(
        values=shap_values[0] if isinstance(shap_values, list) else shap_values[0],
        base_values=explainer.expected_value if hasattr(explainer, 'expected_value') else 0,
        data=X.iloc[0].values,
        feature_names=X.columns.tolist()
    ), show=False)
```

Cell 6: LIME Analysis (runs automatically)

```
# =====
# STEP 5: LIME LOCAL EXPLANATIONS
# This cell runs automatically. Shows why ONE specific decision was made.
# Do not modify unless you know what you're doing.
# =====

import lime
import lime.lime_tabular

# Define a prediction function for LIME
def predict_fn(data):
    """LIME calls this with numpy arrays. Return prediction probabilities."""
    df = pd.DataFrame(data, columns=X.columns)
    if hasattr(user_model, 'predict_proba'):
        return user_model.predict_proba(df)
    else:
        # For API-based solutions, call the API
        results = df.apply(lambda row: [0, 1] if call_ai_api(row) == "Positive" else [1, 0], axis=1)
        return np.array(results.tolist())

feature_names = X.columns.tolist()
class_names = ["Negative", "Positive"]

explainer = lime.lime_tabular.LimeTabularExplainer(
    training_data=X.values,
    feature_names=feature_names,
    class_names=class_names,
    mode='classification'
)

exp = explainer.explain_instance(
    X.iloc[0].values,
    predict_fn,
    num_features=5
)

print("=== LIME Explanation for Sample #1 ===")
exp.show_in_notebook()
```

Cell 7: Fairness Audit (runs automatically)

```
# =====  
# STEP 6: FAIRNESS AUDIT  
# Tests whether your solution treats different groups equally.  
# You need to identify at least one sensitive attribute (e.g., gender, age).  
# =====  
from fairlearn.metrics import demographic_parity_difference, equalized_odds_difference  
  
# TODO: Replace with your sensitive attribute column name  
sensitive_feature = "your_sensitive_feature_column"  
  
if sensitive_feature in X.columns:  
    sensitive = X[sensitive_feature]  
    pred_values = predictions if hasattr(predictions, 'values') else np.array(predictions)  
  
    print("=== Fairness Audit ===")  
    dp = demographic_parity_difference(y, pred_values, sensitive_features=sensitive)  
    eo = equalized_odds_difference(y, pred_values, sensitive_features=sensitive)  
    print(f"Demographic Parity Difference: {dp:.4f} {'✓' if abs(dp) < 0.1 else '⚠'}")  
    print(f"Equalized Odds Difference: {eo:.4f} {'✓' if abs(eo) < 0.1 else '⚠'}")  
else:  
    print(f"⚠ Sensitive feature '{sensitive_feature}' not found in dataset.")  
    print("Identify a protected attribute (gender, age, region, etc.) and update the variable above")
```


Cell 8: Counterfactual Demo

```

# =====
# STEP 7: COUNTERFACTUAL ANALYSIS
# Shows minimum changes needed to flip the prediction.
# =====

import numpy as np

def get_prediction_for_row(row):
    """Get prediction for a single row."""
    if hasattr(user_model, 'predict'):
        return user_model.predict([row.values])[0]
    else:
        return call_ai_api(row)

def counterfactual_for_sample(model_or_fn, sample, feature_names, target_class=1):
    """Find minimum changes needed to flip prediction to target_class."""
    sample = sample.copy()
    current_pred = get_prediction_for_row(sample)

    if current_pred == target_class:
        return f"Already predicted as class {target_class}. No changes needed."

    # Try changing one numeric feature at a time
    for feat in feature_names:
        original = sample[feat]
        if not np.issubdtype(type(original), np.number):
            continue
        for pct in [0.1, 0.25, 0.5, 1.0]:
            sample[feat] = original * (1 + pct)
            new_pred = get_prediction_for_row(sample)
            if new_pred == target_class:
                return f"Change '{feat}' from {original:.2f} to {sample[feat]:.2f} (+{pct*100:.0f}%)
            sample[feat] = original

    return "No single-feature change was sufficient to flip prediction."

# Test on first sample
result = counterfactual_for_sample(user_model, X.iloc[0], X.columns.tolist())
print(f"Counterfactual: {result}")

```

Cell 9: Generate Model Card

```

# =====
# STEP 8: GENERATE MODEL CARD
# Outputs a standardized markdown report of your solution's explainability.
# If the modelcard package is not installed, uses a simple fallback.
# =====
try:
    from modelcard import generate_card
    card = generate_card(
        model=user_model,
        X_train=X,
        X_test=X,
        y_test=y,
        model_name=solution_name,
        model_type=tech_stack,
        dataset=dataset_description,
        team=solution_name
    )
    print("Model Card generated using modelcard package")
except ImportError:
    # Fallback: manual Model Card from template
    from datetime import datetime
    card = f"""# Model Card – {solution_name}

**Tech Stack:** {tech_stack}
**Dataset:** {dataset_description}
**Date:** {datetime.now().strftime('%Y-%m-%d')}
**Target:** {target_variable}

## Performance
- Samples tested: {len(X)}
- Accuracy: [fill in]
- Predictions collected: {len(predictions)}

## Top Features (SHAP)
[Review the SHAP summary plot above and list your top 3]
1. [Feature]: [why it matters]
2. [Feature]: [why it matters]
3. [Feature]: [why it matters]

```

```

## Fairness Results
- Demographic Parity: [fill in from Cell 6]
- Equalized Odds: [fill in from Cell 6]

## Counterfactual
{result}

## Limitations
- [List what your solution struggles with]
- [List any bias or fairness concerns found]
"""

    print("Model Card generated using fallback template (modelcard package not installed)")

print(card)

# Save to file
import os
filename = f"{solution_name.replace(' ', '_')}_MODEL_CARD.md"
with open(filename, "w") as f:
    f.write(card)
print(f"\nSaved to {filename}")

```

Deliverable B: Evaluation Markdown Templates

B1: Model Card Template

File: templates/model_card_template.md

Model Card – [Solution Name]

****Team:**** [Team Name]
****AI Solution:**** [Describe: API used, model type, no-code tool, etc.]
****Tech Stack:**** [List all AI tools/APIs/libraries used]
****Dataset:**** [Name & Source]
****Date:**** [Submission Date]

1. Solution Overview

Property	Value
---	---
Solution Type	
AI Tools Used	
API/Model Version	
Dataset Size	
Target Variable	
Number of Input Features	

2. Performance

Metric	Value
---	---
Accuracy	
Precision	
Recall	
F1 Score	

3. Explainability Analysis

Tech Choice Justification

Why did your team choose this AI tool/approach? What alternatives did you consider?

Top Feature Drivers (SHAP)

Rank	Feature	Importance	Why it matters
---	---	---	---

	1				
	2				
	3				

Local Explanation Quality (LIME)

- Explanation fidelity for Sample #1: [score]
- Does the local explanation make business sense? [yes/no + why]

Counterfactual Examples

	Sample		Original Prediction		Change Needed		Flipped?	
	---		---		---		---	
	#1							
	#2							

4. Fairness Assessment

	Metric		Value		Threshold		Pass?	
	---		---		---		---	
	Demographic Parity Difference			< 0.1				
	Equalized Odds Difference			< 0.1				

Sensitive Features Tested

	Feature		Groups		Bias Detected?		Discussion	
	---		---		---		---	

5. Solution Limitations

- [] This solution performs poorly when [condition]
- [] The API/model has known biases toward [group]
- [] Prompt-dependent: results change significantly with prompt wording
- [] Other limitations:

6. Stakeholder Notes

****Who should use this solution:**** [describe]

****Who should NOT use this solution:**** [describe]

```
**Regulatory considerations:** [GDPR, HKMA banking circulars, GBA cross-border data policies, etc.]  
**Human oversight required:** [yes/no + describe]
```

```
---
```

Academic Citation

If you use this toolkit in your research, please reference the software repository. Citation confi

B2: Checklist Template

File: templates/checklist_template.md

RAI Pre-Submission Checklist

Complete this checklist before submitting your Pull Request.

Solution Documentation

- ☐ I have documented all AI tools/APIs used in my solution
- ☐ I have justified why these tools were chosen over alternatives
- ☐ I have identified who the end users of this solution are
- ☐ I have identified who would be harmed by incorrect outputs
- ☐ I have identified which regulatory frameworks may apply (GDPR, HKMA banking circulars, GBA c

Explainability

- ☐ I have used SHAP for global feature importance
- ☐ I have used LIME for local (per-prediction) explanations
- ☐ I can explain whether SHAP/LIME are appropriate for my solution type
- ☐ I have generated at least 2 counterfactual examples

Fairness

- ☐ I have identified sensitive/protected attributes in my data
- ☐ I have run Demographic Parity and Equalized Odds checks
- ☐ I have documented any bias found and proposed mitigations

Documentation

- ☐ My Model Card is fully populated (no empty fields)
- ☐ My Streamlit/Gradio app runs without errors
- ☐ My README explains how to reproduce my solution
- ☐ I have included a demo video or screenshots

Code Quality

- ☐ My notebook runs from top to bottom without errors
- ☐ All TODO placeholders have been replaced
- ☐ I have removed any hardcoded API keys or paths
- ☐ I have added comments explaining non-obvious code

`## Citation`

`- [ ] My project includes the citation block for AXAI-Toolkit`

Deliverable C: GitHub Grading Pipeline & PR Template

C1: Pull Request Template

File: `.github/PULL_REQUEST_TEMPLATE.md`

Project Submission – [Team Name]

Project Details

- **Team Name:**
- **Project Title:**
- **AI Solution:** [Describe: API/model/tool used]
- **Dataset Used:**
- **GitHub Members:** @tag1 @tag2 @tag3

Self-Assessment

Functionality

- [] My Colab notebook runs end-to-end without errors
- [] My Streamlit/Gradio app launches and displays results
- [] The Model Card is fully populated with real metrics

Explainability

- [] SHAP global feature importance plots are included
- [] LIME local explanation is demonstrated
- [] At least 2 counterfactual examples are shown
- [] I can explain whether these methods suit my solution type

Fairness

- [] Sensitive attributes identified and tested
- [] Demographic Parity check completed
- [] Equalized Odds check completed
- [] Bias findings documented with proposed mitigations

Documentation

- [] README explains how to reproduce results
- [] Pre-submission checklist completed
- [] Demo video or screenshots included

Instructor Use Only

Criteria	Points	Max	Notes
---	---	---	---
Solution documentation & tech choice justification		15	

SHAP analysis completeness		15		
LIME explanation quality		10		
Counterfactual examples		10		
Fairness audit thoroughness		15		
Dashboard functionality		15		
Model Card completeness		10		
Code quality & documentation		10		
Total		**100**		

****Feedback:****

C2: Grading Rubric

File: templates/grading_rubric.md

XAI Project Grading Rubric

Scoring Breakdown (100 points total)

Solution Documentation & Tech Choice (15 pts)

Score	Criteria
13-15	All AI tools documented, tech choice justified with alternatives considered, clear stakeholder mapping present
9-12	Tools documented, basic justification, stakeholder mapping present
0-8	Missing documentation or no justification for tech choices

SHAP Analysis (15 pts)

Score	Criteria
13-15	Global + local SHAP plots present, interpretation in plain language, top features discussed
9-12	SHAP plots present but minimal interpretation or suitability discussion
0-8	SHAP missing, incorrect, or not suitable for solution type without discussion

LIME Analysis (10 pts)

Score	Criteria
9-10	Local explanation shown for ≥2 samples, fidelity score reported, business sense discussed
6-8	Local explanation shown for 1 sample, no fidelity or business sense discussion
0-5	LIME missing or not suitable for solution type without discussion

Counterfactual Examples (10 pts)

Score	Criteria
9-10	≥3 meaningful counterfactuals with business/real-world interpretation
6-8	2 counterfactuals with basic interpretation
0-5	<2 or counterfactuals don't make sense

Fairness Audit (15 pts)

Score	Criteria
13-15	Both metrics computed, sensitive attributes identified, bias findings discussed with mitigation
9-12	Both metrics computed, basic discussion of findings
0-8	Only one metric or no discussion

Dashboard (15 pts)

Score	Criteria
-------	----------

---	---
-----	-----

13-15	App runs, shows predictions + explanations, counterfactual interaction works, clean UI
-------	--

9-12	App runs but UI is rough or some features broken
------	--

0-8	App doesn't launch or is non-functional
-----	---

Model Card (10 pts)

Score	Criteria
-------	----------

---	---
-----	-----

9-10	All sections populated, accurate metrics, limitations honestly discussed, tech choices justified
------	--

6-8	Most sections populated, some empty fields
-----	--

0-5	Model Card largely empty or copied from template
-----	--

Code Quality & Documentation (10 pts)

Score	Criteria
-------	----------

---	---
-----	-----

9-10	Notebook runs top-to-bottom, clear comments, README reproducible, PR template complete
------	--

6-8	Minor issues running, adequate comments
-----	---

0-5	Notebook has errors, no README, poor code organization
-----	--

Deliverable D: Demo Repo (Complete Worked Example)

Path in repo: `Case_Studies/demo-titanic/`

A complete, end-to-end example using the Titanic dataset so students can see exactly what a finished submission looks like.

Required Files

```
Case_Studies/demo-titanic/
├─ demo-titanic.ipynb      # Full notebook with all cells completed
├─ MODEL_CARD.md          # Completed Model Card
├─ README.md              # How to reproduce this demo
├─ app.py                 # Working Streamlit dashboard
├─ demo.mp4               # 2-minute screen recording walkthrough
└─ requirements.txt        # pip requirements (for local run)
```

What the Demo Must Show

1. **Describe the solution** — "RandomForest classifier on Titanic passenger data"
 2. **Load data** → clean missing values
 3. **Get predictions** → report accuracy
 4. **Run SHAP** → show summary plot + waterfall for one passenger
 5. **Run LIME** → show local explanation for one prediction
 6. **Run Fairness audit** → test Gender and Age as sensitive attributes
 7. **Counterfactual** → "What would need to change for this passenger to survive?"
 8. **Model Card** → fully populated with all sections complete
 9. **Streamlit app** → input passenger details, see prediction + explanation
-

Deliverable E: Setup Guide

File: templates/README.md

A step-by-step guide for students to get started.

Getting Started with the XAI Evaluation Kit

Quick Start

1. Download ``templates/AXAI_Master_Template.ipynb`` from the repo
2. Open it in your preferred environment (Google Colab, Jupyter, Baidu AI Studio)
3. Run Cell 1 (Master Import Cell) – installs all libraries (~2 min)
4. Fill in your solution details in Cell 2
5. Replace the dataset in Cell 3
6. Replace the prediction logic in Cell 4
7. Cells 5-9 run automatically – they explain your solution

Quick Start (Local)

1. Clone this repo: ``git clone https://github.com/RAI-Incubation-Lab/AXAI-Toolkit``
2. Install dependencies: ``pip install -r requirements.txt``
3. Open ``templates/AXAI_Master_Template.ipynb`` in Jupyter
4. Follow the Quick Start steps above

What Each Cell Does

- Cell 1: Installs all libraries (run once, ~2 min)
- Cell 2: Describe your AI solution (you fill this in)
- Cell 3: Load your test data (you fill this in)
- Cell 4: Get predictions from your solution (you fill this in)
- Cell 5: SHAP analysis (auto-runs, don't modify)
- Cell 6: LIME analysis (auto-runs, don't modify)
- Cell 7: Fairness audit (auto-runs, don't modify)
- Cell 8: Counterfactual demo (auto-runs, don't modify)
- Cell 9: Generate Model Card (auto-runs, don't modify)

Different Solution Types

****Scikit-learn / XGBoost model:**** Use PATH A in Cell 4. Everything works out of the box.

****API-based (ChatGPT, Claude, etc.):**** Use PATH B in Cell 4. Write a function that calls your API.

****No-code tool:**** Use PATH C in Cell 4. Export predictions as CSV and load them.

Common Issues

****Error: library not found**** → Re-run Cell 1 (Master Import Cell)

****SHAP error: model type not supported**** → The template automatically falls back to KernelExplainer

****LIME error: feature count mismatch**** → Make sure your training and test data have the same columns

****API rate limits**** → Use a smaller sample (X.iloc[:50]) for explainability analysis

Need Help?

- Check the [\[demo-titanic example\]\(../Case_Studies/demo-titanic/\)](#)
- Ask your TA
- Post in the course discussion

Deliverable F: Model Card Generator Package

Path in repo: `tools/modelcard/`

Lead: Hoper

Status: Runs in parallel with TA deliverables. The template has a fallback (Cell 9) so students are never blocked if this isn't ready yet.

What It Is

A pip-installable Python package (`pip install axai-modelcard`) that automates Model Card generation. When ready, students replace the fallback Cell 9 with:

```
from modelcard import generate_card
card = generate_card(model, X_train, X_test, y_test, ...)
```

This package is the core technical artifact described in the JOSS paper.

Package Structure

```
tools/modelcard/  
├─ __init__.py  
├─ generate.py          # Main generate_card() function  
├─ fairness.py          # Fairlearn/AIF360 integration  
├─ templates/          # Model Card markdown templates  
│   ├─ default.md  
│   └─ edtech.md  
├─ tests/  
│   └─ test_generate.py  
├─ setup.py  
└─ README.md
```

generate_card() Function Signature

```
def generate_card(  
    model,                # trained model or prediction function  
    X_train,              # training features  
    X_test,               # test features  
    y_test,               # test labels  
    model_name: str,      # project name  
    model_type: str,      # "RandomForest", "GPT-4 API", etc.  
    dataset: str,         # dataset description  
    team: str,            # team name  
    sensitive_features=None, # optional: column name for fairness audit  
) -> str:                # returns markdown Model Card
```

JOSS Paper Pipeline

The AXAI-Toolkit will be submitted to the **Journal of Open Source Software (JOSS)**. JOSS is a peer-reviewed open-access journal that publishes papers describing research software. It has a fast review process and assigns a DOI to each accepted paper.

Why JOSS

- **Software counts as research output** — JOSS papers are indexed, DOI-registered, and citable
- **Fast review** — weeks, not months like traditional journals
- **Open source alignment** — the paper describes the software; the review tests it
- **Academic credibility** — peer-reviewed publication for open-source toolkits

Every deliverable we build feeds directly into a section of the JOSS paper:

JOSS Paper Section	What It Needs	Which Deliverable Provides It
Summary	High-level description for non-specialists	Deliverable A (Master Template intro), Deliverable E (README)
Statement of Need	Who uses this, why, and why custom code	Deliverable B (Model Card template — tech choice justification), Deliverable C (grading rubric)
State of the Field	Comparison to existing tools (SHAP, LIME, Alibi)	Deliverable E (Setup Guide — "Common Issues" + alternatives)
API / Core Functionality	Core code description	Deliverable F (<code>generate_card()</code> function, <code>modelcard/</code> package)
Reproducibility	Working example anyone can run	Deliverable D (demo-titanic — complete, runnable)
Installation Instructions	Clean pip install path	Deliverable F (<code>setup.py</code> , <code>requirements.txt</code>), Deliverable E
License	OSI-approved	<code>LICENSE</code> file (MIT or Apache 2.0)
Automated Tests	CI passes	Deliverable F (<code>tests/</code> directory)
Community Guidelines	How to contribute	Deliverable C (PR template shows contribution flow)

Milestone: Once Deliverables D + E + F are complete, the JOSS paper can be drafted from the repo content. No separate paper-writing sprint needed.

Definition of Done

1. A new student can open the Colab template, describe their AI solution, plug in their data and prediction logic, and get a full XAI report in under 10 minutes
2. The demo-titanic example is a complete, polished submission that would score 90+ on the rubric
3. The grading rubric lets the instructor grade any submission in under 15 minutes
4. All files are checked into `github.com/RAI-Incubation-Lab/AXAI-Toolkit` in the correct locations (see repo structure above)
5. The Model Card fallback (Cell 9) produces a usable report even without the modelcard package
6. Every deliverable maps to a JOSS paper section — the paper can be drafted from repo content without a separate writing sprint